

PRoH: Dynamic Planning and Reasoning over Knowledge Hypergraphs for Retrieval-Augmented Generation

Xiangjun Zai
University of New South Wales
Sydney, Australia
xiangjun.zai@unsw.edu.au

Xingyu Tan*
University of New South Wales
Sydney, Australia
xingyu.tan@unsw.edu.au

Xiaoyang Wang
University of New South Wales
Sydney, Australia
xiaoyang.wang1@unsw.edu.au

Qing Liu
Data61, CSIRO
Hobart, Australia
q.liu@data61.csiro.au

Xiwei Xu
Data61, CSIRO
Sydney, Australia
xiwei.xu@data61.csiro.au

Wenjie Zhang
University of New South Wales
Sydney, Australia
wenjie.zhang@unsw.edu.au

Abstract

Knowledge Hypergraphs (KHs) have recently emerged as a knowledge representation for retrieval-augmented generation (RAG), offering a paradigm to model multi-entity relations into a structured form. However, existing KH-based RAG methods suffer from three major limitations: static retrieval planning, non-adaptive retrieval execution, and superficial use of KH structure and semantics, which constrain their ability to perform effective multi-hop question answering. To overcome these limitations, we propose **PRoH**, a dynamic Planning and Reasoning over Knowledge Hypergraphs framework. PRoH incorporates three core innovations: (i) a context-aware planning module that sketches the local KH neighborhood to guide structurally grounded reasoning plan generation; (ii) a structured question decomposition process that organizes subquestions as a dynamically evolving Directed Acyclic Graph (DAG) to enable adaptive, multi-trajectory exploration; and (iii) an Entity-Weighted Overlap (EWO)-guided reasoning path retrieval algorithm that prioritizes semantically coherent hyperedge traversals. Experiments across multiple domains demonstrate that PRoH achieves state-of-the-art performance, surpassing the prior SOTA model HyperGraphRAG by an average of 19.73% in F1 and 8.41% in Generation Evaluation (G-E) score, while maintaining strong robustness in long-range multi-hop reasoning tasks.

CCS Concepts

• Information systems → Question answering.

Keywords

Large Language Models; Retrieval-Augmented Generation; Multi-hop Question Answering; Knowledge Graph

ACM Reference Format:

Xiangjun Zai, Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, and Wenjie Zhang. 2026. PRoH: Dynamic Planning and Reasoning over Knowledge Hypergraphs for Retrieval-Augmented Generation. In *Proceedings of the*

ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792611>

1 Introduction

To improve the factual accuracy and specificity of large language model (LLM) responses, Retrieval-Augmented Generation (RAG) has emerged as a promising approach that integrates external knowledge through the in-context learning capabilities of LLMs. However, traditional RAG systems rely primarily on semantic similarity, fail to capture the structured relational knowledge inherent in many information domains, and often retrieve redundant or noisy content [39]. To address this limitation, Graph-based RAG has been introduced to integrate explicitly structured representations of knowledge into the retrieval process, enabling more accurate and interpretable reasoning [5, 11–13]. By representing entities and their relationships as Knowledge Graphs (KGs), these approaches can capture indirect relations and support multi-hop reasoning across interconnected facts. Nevertheless, most existing Graph-based RAG frameworks model only relations that involve exactly two entities. This design overlooks a fundamental property of real-world knowledge: many relations are inherently n-ary, involving more than two entities simultaneously. As shown in Figure 1, the relation "Mario + Rabbids Kingdom Battle is the first major collaboration between Nintendo and Ubisoft." connects three entities: "Mario + Rabbids Kingdom Battle", "Nintendo" and "Ubisoft". In such cases, the semantics of a relation is established only when all participating entities are considered together. Decomposing these n-ary relations into multiple binary edges inevitably causes a loss of critical structural and semantic information [6, 9, 31, 48].

To address the representational gap, Knowledge Hypergraphs (KHs) have been proposed as a more compact and semantically expressive knowledge structure for Graph-based RAG [4, 7, 23, 42]. KHs generalize standard KGs by allowing hyperedges to connect multiple nodes simultaneously, naturally capturing complex multi-entity interactions and preserving n-ary relational semantics more faithfully. As a result, KHs enhance both the storage of knowledge and its retrieval efficiency for downstream comprehension tasks. Currently, most KH-based RAG systems follow a three-stage pipeline: i) KH construction: extract entities and n-ary relations from the text sources to build the KH; ii) KH-guided retrieval: link query topics with entities and map queries to hyperedges through predefined similarity metrics, followed by a shallow, heuristic-based

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792611>

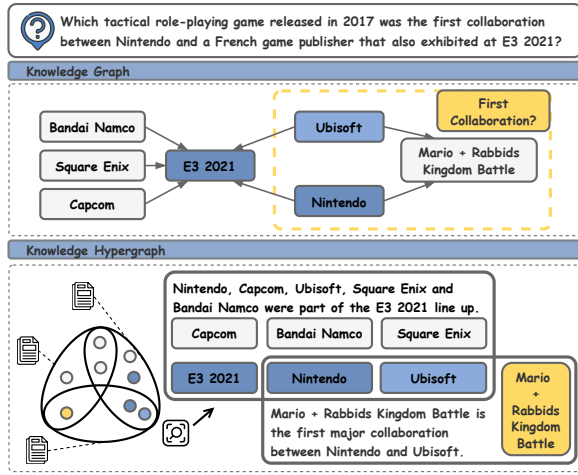


Figure 1: Illustration of Knowledge Hypergraph.

graph fusion that retrieves graph elements along with their corresponding source text chunks; *iii*) Answer generation: pass the retrieved knowledge directly to the generation module. Although this approach has shown promising results in question-answering (QA) tasks, it does not fully exploit the expressive potential of KHs and suffers from the following limitations:

Limitation 1: Static Retrieval Planning. Existing frameworks rely on predefined, hard-coded retrieval pipelines that apply the same sequence of operations regardless of query content or graph context. For example, HGRAG [42] performs a KH diffusion with a fixed number of iterations, without evaluating whether the retrieved passages are sufficient. This rigid design prevents the model from adapting its retrieval plan to the question semantics or the KH topology, leading to inefficient and misaligned knowledge access.

Limitation 2: Non-Adaptive Retrieval Execution. Current systems perform retrieval in a one-shot, non-iterative manner, relying solely on the original query. For instance, HyperGraphRAG [23] identifies and retrieves relevant entities and hyperedges based on a predefined similarity threshold in one graph call. Such static execution fails to refine retrieval using intermediate reasoning results, limiting the system’s capability for multi-hop reasoning.

Limitation 3: Superficial Use of Graph Structure and Semantics. Existing methods primarily treat hyperedges as simple links or routing mechanisms for accessing associated text chunks [7, 23]. This superficial treatment ignores the rich relational semantics encoded in hyperedges and misses the opportunity to guide more precise retrieval and reasoning within KH-based RAG frameworks.

Contribution. To better realize the potential of KHs for RAG, we introduce **PRoH**, a dynamic KH-based RAG framework that fully leverages the expressive power of KHs. PRoH performs structured planning and reasoning directly over KHs, enabling the retriever to adaptively explore and integrate knowledge for multi-hop question answering. The key ideas of PRoH can be summarized as follows: **Context aware planning.** PRoH employs a graph scope-aware planning strategy. Before performing question decomposition, PRoH first sketches the local neighborhood of the topic entities within the

KH. This pre-planning step provides the LLM with a brief yet informative view of the topological and semantic scope of the question-relevant subgraph, mitigating the mismatch between linguistic-only decomposition and the actual graph knowledge available. Consequently, the LLM produces reasoning plans that are more feasible and better aligned with the structure of the underlying KH.

Structured question decomposition with iterative refinement.

PRoH adopts a structured question decomposition approach to explicitly capture the dependencies among subquestions. Instead of treating subquestions as a linear sequence, the reasoning plan is represented as a Directed Acyclic Graph (DAG) that captures logical precedence among them. As reasoning progresses following the topological order of the subquestions, the DAG is iteratively refined. To be more specific, later subquestions are updated, and new subquestions and dependencies may emerge. This mechanism allows the reasoning plan to evolve dynamically and remain consistent with the current reasoning state. PRoH also introduces a state-space search mechanism that performs reasoning as a branching exploration from the current reasoning state, effectively modeling the process as a tree. Unlike prior methods that assume each subquestion has one single correct answer, our approach allows multiple candidate answers per subquestion. That is, several reasoning trajectories can coexist. This design corresponds to the multi-entity nature of n -ary relations, allowing PRoH to manage ambiguity and recover from local errors. The state exploration continues until one or more trajectories reach a goal state, where all subquestions are resolved and a final answer can be derived.

EWO-guided reasoning path retrieval. PRoH employs a fine-grained reasoning path retrieval strategy guided by the Entity-Weighted Overlap (EWO) score, specifically designed for KHs. When visiting a hyperedge, for each hyperedge that overlaps with the current one, the retriever evaluates how strongly the overlapping entities contribute to answering the current subquestion and aggregates these relevance scores to determine the next traversal direction. This process allows the retriever to prioritize semantically meaningful connections rather than relying on purely structural overlaps. As a result, the retrieved reasoning paths are better aligned with the underlying semantics of the question, enabling more accurate and interpretable multi-hop reasoning. In summary, the main contributions of this paper are as follows:

- We propose PRoH, a dynamic KH-based RAG framework that fully leverages the expressive power of hypergraphs for multi-hop question answering.
- We introduce a context-aware planning mechanism that sketches the underlying KH and generates feasible reasoning plans.
- We develop an EWO-guided reasoning path retrieval strategy for fine-grained, semantically aware exploration of KHs.
- PRoH consistently achieves better performance and interpretability than the state-of-the-art HyperGraphRAG framework across multiple knowledge domains, surpassing it by an average of 19.73% in F1 and 8.41% in Generation Evaluation (G-E).

2 Related Work

Graph-based RAG. Graphs are a natural representation for modeling relational structure among entities [10, 33, 40, 45, 50, 51].

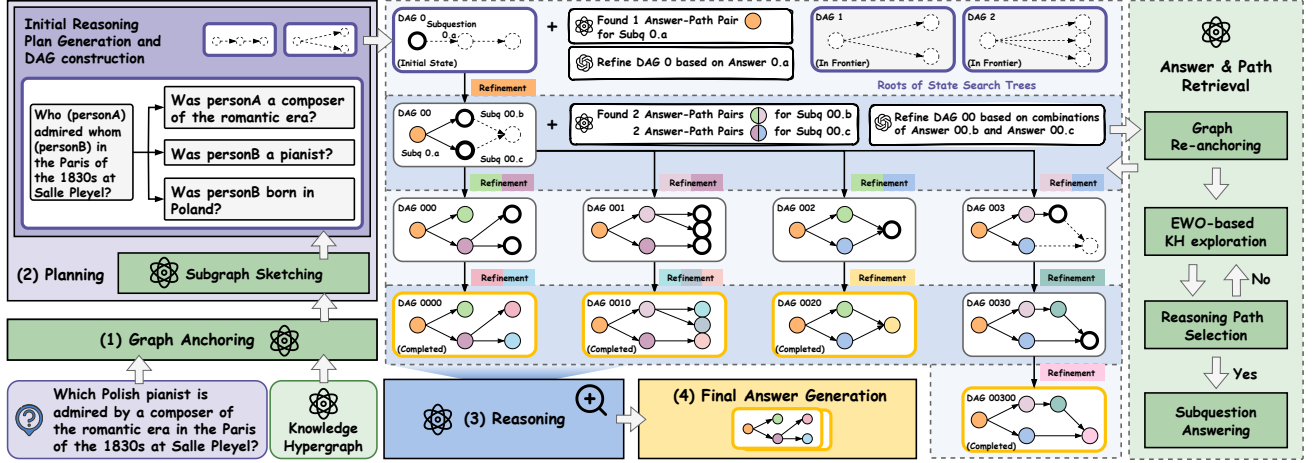


Figure 2: Overview of the PProH framework. (1) Given an input question and a KH, *Graph Anchoring* constructs a question subgraph. (2) The following *Planning* stage sketches the question subgraph and obtains context to generate initial reasoning plans and construct reasoning DAGs for the question. (3) The initial reasoning DAGs serve as roots of the state search trees in the *Reasoning* stage. At each state within the tree, the model processes the current reasoning DAG, retrieves answer-path pairs from the KH and iteratively refines the current DAG to transition to the next state until one or more DAGs are completed. (4) The completed DAGs are then passed to the *Final Answer Generation* as the retrieved knowledge for producing the final answer.

Unlike traditional RAG systems, which rely on flat textual corpora as their primary knowledge source, Graph-based RAG leverages structured representations of relational knowledge. These systems either construct knowledge graphs from documents or utilize existing large-scale graphs such as Wikidata [41] and Freebase [1], enabling more precise retrieval and relation-aware reasoning [15–18, 30, 46, 52, 53]. EmbedKGQA [32] first introduces KG embedding methods into multi-hop KGQA. Recent representative frameworks [5, 11, 12, 49, 54, 62] extract entities and relations from unstructured text using LLMs and index them for retrieval via graph traversal or embedding-based matching. More advanced approaches [3, 8, 21, 22, 26, 27, 34, 37, 60, 61] incorporate iterative reasoning and adaptive planning mechanisms to enhance graph-guided inference. Learning-based methods [24, 28, 36, 38, 56] have also shown promising results. Despite these advances, existing graph-based RAG frameworks remain limited by their binary relational representations [39], which restrict their capacity to model multi-entity facts and capture higher-order relational semantics.

Knowledge Hypergraph-based RAG. Hypergraphs are widely adopted to model higher-order relationships [25, 35, 43, 44, 47, 55, 59]. Early systems such as HyperGraphRAG [23] and HyperRAG [7] extract hyperedges from textual sources to capture higher-order relations and employ similarity-based retrieval to identify relevant entities, hyperedges, and text chunks for question answering. IdepRAG [4] leverages the hypergraph structure by performing Personalized PageRank over topic entities to locate contextually relevant evidence. Meanwhile, HGRAG [42] introduces a cross-granularity retrieval mechanism that integrates fine-grained entity similarity with coarse-grained passage similarity through hypergraph diffusion, effectively balancing structural precision and semantic coherence. However, current hypergraph-based approaches still rely on heuristic, one-shot retrieval pipelines and lack

context-aware and iterative reasoning capabilities, motivating the framework proposed in this study.

3 Preliminaries

Let $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ denote a knowledge hypergraph, where $\mathcal{V}$ and $\mathcal{E}$ represent the set of entities and hyperedges, respectively. Each hyperedge $e \in \mathcal{E}$ links a set of entities $V(e) \subseteq \mathcal{V}$, i.e., $V(e) = \{v_1, v_2, \dots, v_n\}$ where $n \geq 1$. An n -ary relationship fact is modeled as $f = (e, V(e))$. We denote the set of hyperedges in $\mathcal{E}$ that contains an entity v as $E(v)$, i.e., $E(v) = \{e | v \in V(e) \wedge e \in \mathcal{E}\}$. A subgraph $\mathcal{H}_S = (\mathcal{V}_S, \mathcal{E}_S)$ is an induced subgraph of $\mathcal{H}$, if $\mathcal{E}_S \subseteq \mathcal{E}$, $\mathcal{V}_S = \{v | v \in V(e) \wedge e \in \mathcal{E}_S\}$. Two hyperedges e_i and e_j are connected iff $V(e_i) \cap V(e_j) \neq \emptyset$, i.e., an overlap of entities exists between e_i and e_j . The set of hyperedges connected to hyperedge e is defined as the neighbor edge set $\text{Nbr}(e) = \{e' | V(e') \cap V(e) \neq \emptyset \wedge e' \neq e \wedge e' \in \mathcal{E}\}$.

DEFINITION 1 (HYPERGRAPH-BASED RAG). Given a question q and a knowledge hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, hypergraph-based RAG retrieves question-related knowledge, i.e., a set of facts $\mathcal{F}$ from $\mathcal{H}$ and then generates an answer $a(q)$ based on q and $\mathcal{F}$.

DEFINITION 2 (REASONING PATH). Given a knowledge hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, a reasoning path within $\mathcal{H}$ is a connected sequence of hyperedges, represented as $\text{path}(e_s, e_t) = [e_1 = e_s, e_2, \dots, e_{l-1}, e_l = e_t]$, where l denotes the length of the path, i.e., $l = |\text{path}(e_s, e_t)|$.

4 Method

In this section, we introduce the framework of PProH. As illustrated in Figure 2, PProH operates in four stages: Graph anchoring, Planning, Reasoning with Graph retrieval, and Final answer generation. It generates and dynamically refines reasoning plans that consist of subquestions decomposed from the original question, and retrieves knowledge from the KHs via exploring reasoning paths.

4.1 Graph Construction and Indexing

KH Construction. We adopt the graph construction method introduced in HyperGraphRAG [23]. Given the documents, the method first extracts hyperedges from text chunks, then identifies entities within these hyperedges, and finally constructs the KH $\mathcal{H} = (\mathcal{V}, \mathcal{E})$. Each entity is represented as $v = (v^{nm}, v^{desc})$, where v^{nm} is the unique entity name and v^{desc} denotes its associated textual description. Each hyperedge is represented as $e = (e^{nm}, V(e), e^{ref})$, where e^{nm} is the unique hyperedge name, $V(e) \subseteq \mathcal{V}$ is the set of entities linked by e , and e^{ref} is the reference to the originating text chunks. For efficient retrieval, vector databases are maintained for entity names, entity descriptions, and hyperedge names.

Synonym Hyperedge Augmentation. In the original method [23], entity de-duplication relies on exact name matching, which results in isolated hyperedges and weakens the connectivity of the constructed KH. To better utilize graph structure in the later retrieval stage, inspired by HippoRAG2 [13], we introduce synonym hyperedges to the constructed KH. To be more specific, the synonym hyperedges are augmented in three steps. i) For each pair of entities $(v_i, v_j) \in \mathcal{V} \times \mathcal{V}$, we compute the cosine similarity

$$\text{sim}(v_i, v_j) = \cos(\mathbf{z}(v_i^{nm}), \mathbf{z}(v_j^{nm})), \quad (1)$$

where $\mathbf{z}(\cdot)$ is the embedding function. We add a similarity edge $e_{sim} = (v_i, v_j)$ if $\text{sim}(v_i, v_j) \geq \tau$. ii) We form the similarity subgraph $\mathcal{H}_{sim} = (\mathcal{V}_{sim}, \mathcal{E}_{sim})$ and compute its connected components $\{C_1, C_2, \dots, C_m\}$, where each $C_i \subseteq \mathcal{V}_{sim}$. iii) For each connected component C_i , we query an LLM with the entity names and descriptions to determine whether they represent synonymous entities. If the set of synonymous entities $V_{syn} \subseteq C_i$ is confirmed, we add a synonym hyperedge e_{syn} which links all entities in V_{syn} .

4.2 Graph Anchoring

Topic Entity Identification. Given a question q , we first utilize an LLM to extract a set of keywords T_w . Each keyword is linked to a topic entity by computing cosine similarity (Eq. (1)) against all entities in $\mathcal{V}$. The highest-scoring entity whose similarity exceeds a threshold θ_v is selected as the topic entity for the keyword, and all selected entities form the topic entity set $\mathcal{T}$.

Target Hyperedge Matching. To exploit the semantic information contained in the question q , we further match related hyperedges from $\mathcal{E}$. For each hyperedge, we compute a similarity score between q and the hyperedge name, following a formulation analogous to the cosine similarity in Eq. (1). The top-ranked hyperedges that satisfy the threshold θ_e form the target hyperedge set $\mathcal{R}$.

Question Subgraph Construction. Once the topic entities $\mathcal{T}$ and target hyperedges $\mathcal{R}$ are identified, we construct a question subgraph to constrain subsequent retrieval during planning and reasoning. Specifically, for each $v \in \mathcal{T}$ and $e \in \mathcal{R}$, we extract its $D_{\max}$ -hop neighborhood from $\mathcal{H} = (\mathcal{V}, \mathcal{E})$. The question subgraph $\mathcal{H}_q$ is defined as the union of these neighborhoods. We also merge synonymous entities during this subgraph construction phase to obtain a compact representation of the original KH, which benefits the subsequent planning and reasoning.

4.3 Planning

For multi-hop questions, directly retrieving graph elements from the immediate neighborhood of topic entities or target hyperedges is often insufficient. However, naively expanding to deeper neighborhoods quickly leads to an information explosion, as the number of reachable entities and hyperedges grows exponentially with depth. This effect is particularly critical in hypergraphs, where n -ary relations link multiple entities within one single hyperedge, allowing one hyperedge to connect to many hyperedges and thereby rapidly increasing the branching width of the search. Therefore, to control the knowledge retrieval process and selectively retrieve only the most relevant and necessary knowledge from the KH, we introduce the concept of reasoning plans.

DEFINITION 3 (REASONING PLAN). *Given a question q , a reasoning plan is a structured decomposition of q represented as a pair $(\mathcal{Q}, \mathcal{L})$. Here, $\mathcal{Q} = \{q_1, q_2, \dots, q_m\}$ denotes the set of subquestions, where each q_i addresses a partial aspect of q , and $\mathcal{L} \subseteq \mathcal{Q} \times \mathcal{Q}$ encodes dependency constraints among them. The relations $\mathcal{L}$ defines a partial order, that is, if $(q_i, q_j) \in \mathcal{L}$, then q_i must be answered before q_j .*

Question Subgraph Sketching. While it is possible to generate reasoning plans directly from the internal knowledge of an LLM, such plans are often misaligned with the underlying KH. In particular, due to limited awareness of domain-specific relations, the LLM may introduce subquestions that cannot be resolved. Nor is it sufficient to rely solely on the anchored graph elements, namely, topic entities $\mathcal{T}$ and target hyperedges $\mathcal{R}$, to form the plan context, as these elements do not reflect the broader relational structure required for multi-hop reasoning. To mitigate this issue, we construct a plan context graph that efficiently sketches the structure of the question subgraph. This is achieved by treating $\mathcal{T}$ and $\mathcal{R}$ as seeds for controlled exploration. The resulting subgraph provides explicit grounding for plan generation and improves the alignment between the reasoning plan and the available knowledge.

DEFINITION 4 (PLAN CONTEXT GRAPH). *Given a question q , the question subgraph $\mathcal{H}_q = (\mathcal{V}_q, \mathcal{E}_q)$, topic entity set $\mathcal{T} \subseteq \mathcal{V}_q$, target hyperedge set $\mathcal{R} \subseteq \mathcal{E}_q$, and a plan depth d_p , the plan context graph $\mathcal{H}_p = (\mathcal{V}_p, \mathcal{E}_p)$ is defined as a subgraph of $\mathcal{H}_q$, where $\mathcal{V}_p$ and $\mathcal{E}_p$ include entities and hyperedges that are highly relevant to q and are within the d_p -hop neighborhood of either $\mathcal{T}$ or $\mathcal{R}$.*

We initialize the plan graph $\mathcal{H}_p = (\mathcal{V}_p, \mathcal{E}_p)$ using target hyperedges $\mathcal{R}$ and the hyperedges incident to the topic entities $\mathcal{T}$. These hyperedges also serve as the initial frontier for exploration. To guide the search direction, we employ a hyperedge scoring strategy. For each frontier hyperedge e , we first compute entity-level relevance scores for all entities $v \in e$ with respect to the question q :

$$\text{SE}(v \mid q) = \cos(\mathbf{z}(v^{desc}), \mathbf{z}(q)), \quad (2)$$

where $\mathbf{z}(\cdot)$ denotes the embedding function. For each neighboring hyperedge $e' \in \text{Nbr}(e)$, we apply an aggregation function $\text{AGG}(\cdot)$ to combine the scores of the overlapping entities $V(e) \cap V(e')$, producing a hyperedge-level relevance score with respect to q and e . Details are provided in Appendix A.1.

$$\text{SH}(e' \mid q, e) = \text{AGG}(\{\text{SE}(v \mid q) \mid v \in V(e) \cap V(e')\}). \quad (3)$$

Based on the hyperedge-level relevance scores, low-scoring directions (hyperedges) are pruned, and the exploration will focus on directions that are supported by highly relevant entities.

Initial Reasoning Plan Generation. After constructing the plan context graph $\mathcal{H}_p$, we leverage it as a structured input for the LLM to propose reasoning plans. We transfer the graph structure into a natural language plan context by augmenting the hyperedges layer by layer, from the nearest neighborhood of the anchored graph elements to the more distant neighborhoods. This ensures that the context reflects local relevance and progressively broader structural information. Formally, given the plan context graph $\mathcal{H}_p$, the plan context c_p is defined as $c_p = \text{FormPlanContext}(\mathcal{H}_p, \mathcal{T}, \mathcal{R})$, where FormPlanContext denotes the procedure that extracts and formats the relevant subgraph structure into plan context. The LLM is then prompted with the question q , the topic entities $\mathcal{T}$, and the plan context c_p to generate initial reasoning plans.

Initial Reasoning DAG construction. Given a reasoning plan $(Q, \mathcal{L})$, a dependency $(q_i, q_j) \in \mathcal{L}$ is transitively covered if there exists $q_k \in Q$ such that $(q_i, q_k) \in \mathcal{L}$ and $(q_k, q_j) \in \mathcal{L}$. To obtain a minimal representation of $\mathcal{L}$, we apply a Hasse Reduction that removes all dependency relations that can be transitively covered. The resulting reduced relations $\mathcal{L}_H$ retains only immediate (non-transitive) dependencies between subquestions. With $\mathcal{L}_H$, we formally define the reasoning DAG as follows.

DEFINITION 5 (REASONING DAG). *A reasoning DAG is the graph abstraction of a reasoning plan $(Q, \mathcal{L})$. It is defined as a directed acyclic graph (DAG) $D = (Q, \mathcal{L}_H)$, where each node represents a subquestion $q_i \in Q$ and each directed edge $(q_i, q_j) \in \mathcal{L}_H$ encodes the immediate dependency between q_i and q_j .*

An example reasoning DAG is shown in Figure 2. For each initial reasoning plan, we construct a corresponding reasoning DAG and apply a topological sort on the reduced dependency relations $\mathcal{L}_H$ to obtain an execution order over the subquestions. This order will guide the level-by-level completion of the reasoning DAG. The processed reasoning DAGs form the initial reasoning DAG set $\mathcal{D}_0$.

4.4 Reasoning

Once the initial reasoning DAGs are constructed, the next step is to query the KH under their guidance. More specifically, for a reasoning DAG D , questions at the first level without dependencies are resolved first. The retrieved answers are used to refine the current reasoning DAG D , and the questions of the next level will be unlocked for reasoning. This iterative process repeats until all subquestions are answered. The step answers to subquestions and supporting knowledge are stored in the completed reasoning DAG. We refer to this process of progressively resolving and refining DAGs as reasoning. Due to space limitations, the pseudo-code of the reasoning algorithm is provided in Appendix A.2.

DEFINITION 6 (COMPLETED REASONING DAG). *A completed reasoning DAG is defined as a DAG $D_{\text{comp}} = (Q, \mathcal{L}_H, \mathcal{AP})$, where each node $q_j \in Q$ is associated with a non-empty set of answer-path pairs $\mathcal{AP}[q_j] = \{(a_j, p_j)\}$. Here, a_j is a candidate answer to q_j and p_j is a supporting reasoning path in $\mathcal{H}_q$.*

Reasoning as State-Space Search. To explicitly model the reasoning process, we introduce the concept of a reasoning state. A reasoning state records a reasoning DAG and the current progress in resolving its subquestions. An initial state corresponds to an initial reasoning DAG $D \in \mathcal{D}_0$ with no subquestions resolved. A goal state corresponds to a completed reasoning DAG.

DEFINITION 7 (REASONING STATE). *A reasoning state is a pair (D, i) , where $D = (Q, \mathcal{L}_H, \mathcal{AP})$ is a reasoning DAG and i is the index of the current reasoning level. A transition from (D, i) to $(D', i + 1)$ occurs once all subquestions in the i -th level of D , denoted as Q_i , are resolved with non-empty sets of answer-path pairs $\mathcal{AP}_i$.*

Given the reasoning state defined above, we formulate the reasoning process as a structured search over reasoning states and solve it using Depth-First Search (DFS). The frontier $\mathcal{F}$ is initialized with the set of initial reasoning DAGs $\mathcal{D}_0$. At each iteration, a state (D, i) is popped from $\mathcal{F}$. If D is incomplete, the subquestions Q_i at the current level i are attempted. Each subquestion is resolved by retrieving reasoning paths from the anchored graph elements in the query subgraph $\mathcal{H}_q$, within a KH exploration depth limit d_{max} . Details of the retrieval procedure are provided in Section 4.5. The retrieved answer-path pairs are then used to generate successor states. These successor states are pushed back into the frontier for later iterations. If D is completed, it is added to the solution set $\mathcal{D}_{\text{comp}}$. The search terminates once $|\mathcal{D}_{\text{comp}}| \geq K$ or the frontier is empty, where K is a configurable parameter denoting the maximum number of solutions. Formally, the search procedure is summarized as $\mathcal{D}_{\text{comp}} = \text{Reasoning}(\mathcal{D}_0, \mathcal{H}_q, d_{\text{max}}, b, K)$.

State Transitions. Given a state (D, i) , the sets of answer-path pairs $\mathcal{AP}_i$ at level i , we illustrate the state transition process as follows. Since each subquestion $q_j \in Q_i$ is associated with a set of candidate assignments in the form of answer-path pairs AP_j , a valid joint assignment for level i can be obtained by selecting one answer-path pair $(a_j, p_j) \in AP_j$ for every $q_j \in Q_i$. If multiple joint assignments exist, the current state branches accordingly. For each joint assignment $\mathcal{AP}$, a successor reasoning DAG is constructed as $D_{\text{new}} = \text{LLMGenerateNewDAG}(D, \mathcal{AP})$. Here, the LLM is prompted with the current set of subquestions Q together with one candidate answer for each completed subquestion. A refined reasoning plan is proposed by the LLM, which is then validated against the existing reasoning DAG D , and then merged with D to produce the successor reasoning DAG D_{new} .

4.5 Answer and Path Retrieval

The core subroutine for resolving subquestions is the retrieval of answer-path pairs from the question subgraph $\mathcal{H}_q = (\mathcal{V}_q, \mathcal{E}_q)$. We employ beam search to progressively explore $\mathcal{H}_q$ with increasing depth. At each depth level, the subquestion is attempted using the knowledge retrieved based on a reasoning path discovered so far. The process terminates once a reasoning path provides sufficient knowledge to answer the subquestion. Due to space limitations, we detail the algorithm of answer and path retrieval in Appendix A.3.

Graph Re-anchoring. For a subquestion q_j , the relevant knowledge should be within a more concise subregion of the question subgraph $\mathcal{H}_q$. Accordingly, we re-anchor the graph, identify the topic entities $\mathcal{T}_j$ and target hyperedges $\mathcal{R}_j$ specific to q_j , following

the same procedure described in Section 4.2. $\mathcal{T}_j$ and $\mathcal{R}_j$ serve as seeds for the subsequent beam search, i.e., we initialize the search frontier $\mathcal{F}_e$ with $\mathcal{R}_j$ and the hyperedges incident to $\mathcal{T}_j$. The key challenge is then to guide the search toward discovering knowledge most relevant to the question q_j . To address this challenge, we design a strategy tailored to KHs.

EWO-based Search Direction Selection. In standard graphs, two adjacent edges can share at most a single node, in contrast, hyperedges in a hypergraph may overlap on multiple entities. Moreover, the contribution of these overlapping entities to answering a question is not uniform: some are irrelevant, while others provide critical evidence. Thus, neighboring hyperedges should neither be treated as equally relevant, nor should their relevance be determined solely by the number of shared entities. To address this, we propose the **Entity-Weighted Overlap (EWO)** score, a fine-grained strategy in which the relevance of a hyperedge is computed by aggregating the question-specific relevance of its overlapping entities $V(e) \cap V(e')$. We now describe the two-step procedure for computing EWO score.

Entity scoring. Each overlapping entity v is first assigned a provisional relevance score with respect to q_j using the embedding-based similarity score defined in Eq. (2). Entities with similarity scores above the threshold θ_{emb} are retained and further evaluated by the LLM to obtain a finer-grained relevance score. Entities with similarity scores below the threshold θ_{emb} are assigned a relevance score of 0. This score reflects how semantically relevant v is for q_j .

$$\text{EW}(v \mid q_j) = \begin{cases} \text{LLMScore}(v, q_j), & \text{if } \text{SE}(v \mid q_j) \geq \theta_{\text{emb}}, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Hyperedge scoring. The hyperedge score integrates average and maximum scores over overlapping entities. The average score reflects overall semantic relevance, while the maximum score emphasizes salient entities providing strong evidence. This design captures both coarse-grained support and fine-grained discriminative cues.

$$\text{EWO}(e' \mid q_j, e) = \text{AGG}\left(\{\text{EW}(v \mid q_j) \mid v \in V(e) \cap V(e')\}\right). \quad (5)$$

With the EWO score, we now determine where to expand the search within the candidate search directions (partial reasoning paths) F_{cand} . In the first stage, F_{cand} is ranked according to the hyperedge-level EWO scores of terminal hyperedges. From the resulting top-ranked directions, we prompt an LLM to select the top- b directions $F_{\text{sel}} = \text{LLMSelectDirections}(F_{\text{cand}}, q_j, b)$. This evaluates the partial reasoning paths in context, rather than relying solely on the EWO score of the terminal hyperedge.

Reasoning Path Selection. At each depth, we construct a set of candidate reasoning paths P_{cand} from the partial paths $\mathcal{P}$ explored so far. Each candidate reasoning path is then ranked using a path-level relevance score that aggregates the EW scores of entities along the path. Formally, the path-level score is defined as:

$$\text{SP}(p) = \text{sum}\{\text{EW}(v \mid q_j) \mid v \in e \wedge e \in p\} \quad (6)$$

The top-ranked candidate reasoning paths are then passed to the LLM, to determine whether one or more of them provide sufficient knowledge to yield a step answer for q_j . The reasoning paths with sufficient knowledge are $P_{\text{sel}} = \text{LLMSelectPaths}(P_{\text{cand}}, q_j)$.

Step Answer for Subquestion. Once P_{sel} contains valid reasoning paths, we terminate the beam search and attempt to answer sub-question q_j , i.e., the exploration beyond the current depth will not be performed. For each selected path $p_j \in P_{\text{sel}}$, we construct the context for q_j by combining three components: i) the path itself, ii) the descriptions of the entities covered by the path, and iii) the originating text chunks of the hyperedges along the path. Using the fused context c_j as input, an LLM is invoked to produce the step answer $a_j = \text{LLMAnswerStep}(c_j, q_j)$.

4.6 Final Answer Generation

As discussed in Section 4.4, the reasoning module produces a solution set $\mathcal{D}_{\text{comp}}$, where multiple reasoning plans (DAGs) are executed to completion with all sub-questions answered. For each completed DAG $D_{\text{comp}} = (Q, \mathcal{L}_H, \mathcal{AP})$, we aggregate the retrieved knowledge along its reasoning process following dependency relations $\mathcal{L}_H$ and uses this aggregated knowledge to generate a candidate answer $a(q)$ to the original question q . An LLM-based evaluation agent is then introduced to assess these candidate answers $\mathcal{A}(q)$. This judge evaluates each candidate answer $a(q)$ according to its consistency with the corresponding reasoning path, ultimately selecting the top-ranked answer as the final answer $a^*(q)$.

5 Experiments

Experimental Settings. We evaluate PRoH on the KHQA datasets introduced in HyperGraphRAG [23], which span five knowledge domains: Medicine, Agriculture, CS, Legal, and Mix. Since the questions in the KHQA datasets are constructed from sampled knowledge fragments located only 1-3 hops away. We further extend these datasets with long-range questions to better assess the multi-hop reasoning capability of PRoH. Specifically, we generate 200 additional questions per domain using knowledge fragments 3-6 hops away. We also develop and evaluate PRoH-L, a variant of PRoH that employs a fully embedding-based EWO score and uses only the hyperedges along the reasoning paths as context for answer generation. Following [23], we adopt three evaluation metrics: F1, Retrieval Similarity (R-S), and Generation Evaluation (G-E). Due to the space limitation, experiment details, including baselines and implementation details, are provided in Appendix C.

5.1 Main Results

(RQ1) Does PRoH outperform other methods? As shown in Table 1, PRoH achieves state-of-the-art performance across all domains in terms of F1 and G-E scores, outperforming the previous SOTA baseline HyperGraphRAG by an average of 19.73% and up to 22.85% in F1 on the CS domain, as well as by an average of 8.41% and up to 9.67% in G-E. For the R-S score, PRoH generally achieves comparable results to HyperGraphRAG, with up to a 4.75% improvement in the Legal domain. The main weakness appears in the Mix domain, which is reasonable since it integrates knowledge from multiple domains. Unlike HyperGraphRAG, which prioritizes retrieving text with high semantic similarity, PRoH retrieves knowledge that contributes directly to reasoning toward the answer, even when such knowledge is not semantically similar to the surface context. Notably, the Mix domain is also where PathRAG, another

Table 1: Results of PProH across different domains, compared with the state-of-the-art (SOTA). The highest scores are highlighted in bold, while the second-best results are underlined for each dataset.

Method	Medicine			Agriculture			CS			Legal			Mix		
	F1	R-S	G-E	F1	R-S	G-E	F1	R-S	G-E	F1	R-S	G-E	F1	R-S	G-E
LLM-only	12.89	0	43.27	12.74	0	46.85	18.65	0	48.87	21.64	0	49.05	16.93	0	45.65
StandardRAG	27.90	62.57	55.66	27.43	45.81	57.10	28.93	48.40	56.89	37.34	51.68	59.97	43.20	47.26	64.62
PathRAG	14.94	53.19	44.06	21.30	42.37	52.48	26.73	41.89	54.13	31.29	44.03	55.36	37.07	33.73	59.11
HippoRAG2	21.34	59.52	49.57	12.63	18.58	44.85	17.34	23.99	47.87	18.53	34.42	45.93	21.53	18.42	46.35
HyperGraphRAG	35.35	70.19	59.35	33.89	<u>62.27</u>	59.79	31.30	<u>60.09</u>	57.94	43.81	60.47	63.61	48.71	68.21	66.90
PProH-L	45.63	<u>70.84</u>	59.90	<u>50.47</u>	64.28	<u>63.13</u>	46.61	60.79	<u>60.17</u>	51.40	64.58	63.71	<u>53.81</u>	<u>59.32</u>	61.32
PProH	52.94	74.02	67.35	56.67	58.88	69.46	54.15	57.72	66.79	58.81	65.22	69.88	69.16	59.86	76.17

Table 2: Ablation study results (F1).

Method	Agriculture	CS	Mix
PProH	58.49	59.47	69.39
w/o EWO Guide	53.22	56.27	65.63
w/o Synonym Merge	53.26	54.96	64.74
w/o Plan Context	53.70	53.67	64.59
w/o Src Chunks	53.55	54.75	63.76
w/o Target Hyperedge	53.27	55.60	60.81
w/o ALL	51.93	51.96	55.84

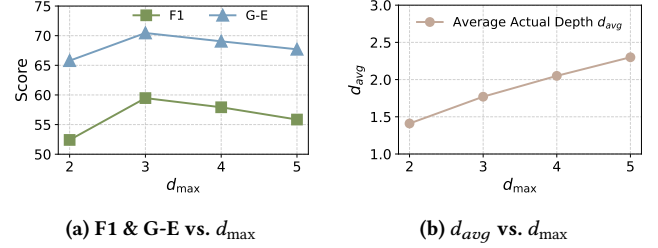
reasoning-path-based retrieval method, attains its lowest R-S score, indicating a similar behavior pattern. For the variant PProH-L, it surpasses HyperGraphRAG by an average of 10.97% and up to 16.58% in F1 on the Agriculture domain. It also remains competitive with HyperGraphRAG in terms of both G-E and R-S scores, showing up to 3.34% improvement in the Agriculture domain and 4.11% improvement in the Legal domain, respectively.

5.2 Ablation Study

(RQ2) Does the main component of PProH work effectively?

We conduct an ablation study across three domains to quantify the contribution of each component. From each domain, Agriculture, CS, and Mix, we randomly sample 200 questions and report the F1 score for comparison. As shown in Table 2, removing the EWO Guide Direction Selection decreases F1 by up to 5.3%, demonstrating its effectiveness in guiding the exploration of reasoning paths based on semantic flow from one hyperedge to another. Removing Synonym Merge reduces F1 by up to 5.2%, indicating that this graph-reduction technique benefits planning and reasoning. Removing Plan Context results in a performance drop of up to 5.8%, highlighting the importance of planning context for the feasibility and alignment of the initial reasoning plan. Without Source Chunks, F1 declines by up to 5.6%, suggesting that the source text provides additional context that contributes to more accurate answers. Eliminating Target Hyperedge Matching in graph anchoring yields the largest drop, up to 8.6%, demonstrating the importance of leveraging semantics encoded in hyperedges. When all modules mentioned above are removed (w/o ALL), performance drops sharply by up to 13.6%, indicating their complementary contributions.

(RQ3) How does the KH exploration depth affect the performance of PProH? PProH performs dynamic exploration on


Figure 3: Impact of depth limit on performance.

KH within the depth limit $d_{\max}$. To assess the impact of $d_{\max}$ on performance, we conduct experiments on 200 randomly sampled questions from the CS domain. We vary $d_{\max}$ from 2 to 5 and report the corresponding F1 and G-E scores in Figure 3(a). We also report the actual exploration depth when the step answer for each subquestion is found and the exploration terminates in Figure 3(b). As shown in Figure 3(a), increasing $d_{\max}$ initially improves both F1 and G-E scores, with performance peaking at $d_{\max}=3$. Beyond this depth, a deeper search even introduces a slight degradation in both metrics. This suggests that the additional search depth does not uncover additional useful information and instead introduces redundant context that fails to improve reasoning quality. The trend in the actual exploration depth supports this interpretation. As $d_{\max}$ increases from 2 to 5, the average actual depth d_{avg} grows modestly from 1.41 (at $d_{\max}=2$) to 2.30 (at $d_{\max}=5$). This indicates that PProH rarely needs to utilize the full depth budget, as most subquestions are resolved within a relatively short reasoning path. This behavior can be attributed to two main reasons. *i)* PProH decomposes questions and dynamically refines its reasoning plan, thus simplifying each subquestion and shortening the reasoning path. *ii)* The EWO-guided exploration helps the system identify relevant paths early, minimizing unnecessary exploration.

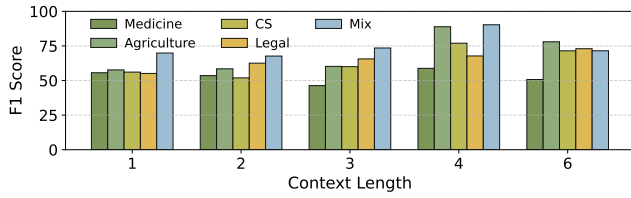
5.3 Effectiveness and Efficiency Evaluation

(RQ4) Does PProH stay effective on long-range multi-hop questions?

We further evaluate PProH on 200 additional long-range questions per domain (3-6 hops). As shown in Table 3, PProH sustains strong performance under these long-range settings, outperforming HyperGraphRAG by an average of 26.68% and up to 44.87% in F1 in the CS domain, as well as by an average of 12.11% and up to 21.28% in G-E. For the R-S score, PProH achieves an average of 1.14% and up to 6.59% improvement in the Legal domain. The

Table 3: Performance of PRoH and baselines on long-range multi-hop QA tasks across domains. The highest scores are highlighted in bold, while the second-best results are underlined for each dataset.

Method	Medicine			Agriculture			CS			Legal			Mix		
	F1	R-S	G-E	F1	R-S	G-E	F1	R-S	G-E	F1	R-S	G-E	F1	R-S	G-E
IO (LLM-only)	23.36	0	0	30.95	0	0	39.27	0	0	43.20	0	0	48.98	0	0
COT (LLM-only)	22.70	0	43.42	34.32	0	52.22	42.41	0	58.16	43.36	0	57.71	50.48	0	62.24
SC (LLM-only)	26.06	0	44.80	37.49	0	53.84	46.42	0	60.34	40.59	0	55.36	54.04	0	63.16
StandardRAG	21.90	66.02	50.13	48.00	50.92	67.81	27.34	59.42	57.11	35.86	55.25	60.37	52.20	59.79	69.91
HyperGraphRAG	39.94	<u>71.72</u>	60.81	52.40	<u>64.36</u>	70.76	29.95	68.78	58.72	38.45	58.30	61.87	<u>64.55</u>	70.23	<u>76.63</u>
PRoH-L	47.30	71.24	60.98	<u>70.02</u>	65.75	<u>74.44</u>	<u>62.11</u>	65.09	<u>69.08</u>	<u>60.25</u>	<u>59.21</u>	67.78	61.19	62.48	66.16
PRoH	51.26	75.11	65.23	81.01	62.22	84.18	74.83	<u>68.44</u>	80.00	71.93	64.89	77.68	79.69	<u>68.45</u>	82.25

**Figure 4: F1 score vs. length of ground-truth context.****Table 4: #Token per question across domains.**

Domain	HyperGraphRAG	PRoH-L	#Token%↓	F1↑
Medicine	21,112	19,732	6.54%	10.28
Agriculture	17,914	12,528	30.07%	16.58
CS	18,666	12,166	34.82%	15.31
Legal	22,086	28,831	-30.54%	7.59
Mix	13,856	9,687	30.09%	5.10

variant PRoH-L, also demonstrates strong performance under these long-range settings, outperforming HyperGraphRAG by an average of 15.11% and up to 32.15% in F1. The robustness of PRoH is also supported by the results in Figure 4, which analyzes the effect of ground truth context length (the number of knowledge fragments used when the question and golden answer are generated) on PRoH's performance. The F1 scores remain stable as the context length grows, suggesting that PRoH can maintain reasoning coherence even when the relevant knowledge spans multiple hops, demonstrating its robustness in long-range, multi-hop reasoning.

(RQ5) Is PRoH-L cost efficient in token consumption? As shown in Table 4, PRoH-L demonstrates notable efficiency in token usage while maintaining competitive performance across all domains. Compared with HyperGraphRAG, PRoH-L significantly reduces the number of tokens used (input + output) per question, with the largest savings observed in the CS domain at 34.82%. Despite the reduced token usage, PRoH-L achieves consistent improvements in F1 score with up to 16.58% in the Agriculture domain. The only exception appears in the Legal domain, where token consumption increases for PRoH-L, however, this increase still yields a positive F1 gain of 7.59%. Overall, these results confirm that PRoH-L achieves a superior balance between efficiency and accuracy, offering a cost-effective alternative to full PRoH.

Table 5: Performance across LLM backbones.

LLM backbone	Agriculture			CS		
	F1	R-S	G-E	F1	R-S	G-E
Qwen3-4B	78.10	63.89	82.26	67.12	67.50	74.03
Qwen3-14B	79.43	64.49	83.41	73.48	67.95	79.16
Qwen3-80B	84.27	62.97	86.42	75.97	69.07	81.87

(RQ6) How does the performance of PRoH vary across different LLM backbones? To study the impact of LLM backbones, we evaluate our method using LLMs of varying model sizes on the long-range questions from the Agriculture and CS domains. As reported in Table 5, stronger LLMs consistently improve F1 and G-E scores across both domains. As Qwen3 scales from 4B to 14B to 80B, the F1 score increases from 78.10% to 79.43% to 84.27% in Agriculture and from 67.12% to 73.48% to 75.97% in CS. This trend indicates that larger models are more effective at planning and retrieving question-relevant knowledge from KHs, enabling more effective evidence integration for high-quality answer generation.

To further evaluate PRoH, we perform additional ablation studies on long-range multi-hop question answering and examine the effects of the state search strategy and planning depth. We also analyze token usage across modules. Detailed results are provided in Appendix B.

6 Conclusion

This paper presents PRoH, a dynamic Knowledge Hypergraph-based RAG framework for multi-hop question answering. By introducing context-aware planning, structured iterative question decomposition and an EWO-guided reasoning path retrieval strategy, PRoH enables adaptive planning and reasoning on knowledge hypergraphs with beyond binary relational structures. Experimental results demonstrate that PRoH achieves SOTA performance across multiple knowledge domains, surpassing the prior SOTA by an average of 19.73% in F1 and 8.41% in G-E score, while maintaining high robustness in long-range multi-hop reasoning tasks.

Acknowledgments

Xiaoyang Wang is supported by the Australian Research Council DP230101445 and DP240101322. Wenjie Zhang is supported by the Australian Research Council DP230101445 and FT210100303.

References

- [1] Kurt D. Bollacker, Colin Evans, Praveen K. Paritosh, Tim Sturge, and Jamie Taylor. 2008. Freebase: a collaboratively created graph database for structuring human knowledge. In *SIGMOD Conference*. ACM, 1247–1250.
- [2] Boyu Chen et al. 2025. PathRAG: Pruning Graph-based Retrieval Augmented Generation with Relational Paths. *arXiv preprint arXiv:2502.14902* (2025).
- [3] Liyi Chen, Panrong Tong, et al. 2024. Plan-on-Graph: Self-Correcting Adaptive Planning of Large Language Model on Knowledge Graphs. In *NeurIPS*.
- [4] Chengfeng Dou, Ying Zhang, Zhi Jin, Wenpin Jiao, Haiyan Zhao, Yongqiang Zhao, and Zhengwei Tao. 2025. Enhancing LLM Generation with Knowledge Hypergraph for Evidence-Based Medicine. *arXiv preprint arXiv:2503.16530* (2025).
- [5] Darren Edge, Ha Trinh, et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [6] Bahare Fatemi, Perouz Taslakian, David Vázquez, and David Poole. 2020. Knowledge Hypergraphs: Prediction Beyond Binary Relations. In *IJCAI*. 2191–2197.
- [7] Yifan Feng, Hao Hu, Xingliang Hou, Shiquan Liu, et al. 2025. Hyper-RAG: Combating LLM Hallucinations using Hypergraph-Driven Retrieval-Augmented Generation. *arXiv preprint arXiv:2504.08758* (2025).
- [8] Junqi Gao, Xiang Zou, et al. 2025. Graph Counselor: Adaptive Graph Exploration via Multi-Agent Synergy to Enhance LLM Reasoning. In *ACL (1)*. 24650–24668.
- [9] Saiping Guan, Xiaolong Jin, Yuanzhuo Wang, and Xueqi Cheng. 2019. Link Prediction on N-ary Relational Data. In *WWW*. ACM, 583–593.
- [10] Qiuyu Guo, Jianye Yang, Wenjie Zhang, Hanchen Wang, Ying Zhang, and Xuemin Lin. 2025. Efficient and Accurate Subgraph Counting: A Bottom-up Flow-learning Based Approach. *Proceedings of the VLDB Endowment* 18, 8 (2025), 2695–2708.
- [11] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. LightRAG: Simple and Fast Retrieval-Augmented Generation. In *Findings of EMNLP*.
- [12] Bernal Jimenez Gutierrez et al. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *NeurIPS*.
- [13] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. In *Forty-second International Conference on Machine Learning, ICML*.
- [14] Huangshuai He, Zhengyi Yang, Dong Wen, et al. 2025. A Survey on Efficient Graph Reachability Queries. In *PAKDD* (6).
- [15] Yizhang He, Kai Wang, et al. 2025. Robust Privacy-Preserving Triangle Counting under Edge Local Differential Privacy. *Proc. ACM Manag. Data* (2025).
- [16] Yizhang He, Kai Wang, Wenjie Zhang, et al. 2023. Scaling Up k-Clique Densest Subgraph Detection. *Proc. ACM Manag. Data* (2023).
- [17] Chengkai Huang, Hongtao Huang, Tong Yu, Kaige Xie, Junda Wu, et al. 2025. A Survey of Foundation Model-Powered Recommender Systems: From Feature-Based, Generative to Agentic Paradigms. *arXiv preprint arXiv:2504.16420* (2025).
- [18] Chengkai Huang, Yu Xia, Rui Wang, Kaige Xie, Tong Yu, Julian McAuley, and Lina Yao. 2025. Embedding-informed adaptive retrieval-augmented generation of large language models. In *COLING*. 1403–1412.
- [19] Fan Li, Xiaoyang Wang, Dawei Cheng, Wenjie Zhang, Ying Zhang, et al. 2024. Hypergraph Self-supervised Learning with Sampling-efficient Signals. In *IJCAI*.
- [20] Fan Li, Xiaoyang Wang, Wenjie Zhang, Ying Zhang, and Xuemin Lin. 2025. DHG-Bench: A Comprehensive Benchmark for Deep Hypergraph Learning. *arXiv preprint arXiv:2508.12244* (2025).
- [21] Xujian Liang and Zhaoquan Gu. 2025. Fast Think-on-Graph: Wider, Deeper and Faster Reasoning of Large Language Model on Knowledge Graph. In *AAAI*.
- [22] Runxuan Liu et al. 2025. Ontology-Guided Reverse Thinking Makes Large Language Models Stronger on Knowledge Graph Question Answering. In *ACL*.
- [23] Haoran Luo, Haihong E, et al. 2025. HyperGraphRAG: Retrieval-Augmented Generation via Hypergraph-Structured Knowledge Representation. In *NeurIPS*.
- [24] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2024. Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning. In *ICLR*.
- [25] Qi Luo, Wenjie Zhang, et al. 2024. Hierarchical structure construction on hypergraphs. In *CIKM*.
- [26] Jie Ma, Zhitaogao, Qi Chai, Wangchun Sun, Pinghui Wang, Hongbin Pei, Jing Tao, Lingyun Song, Jun Liu, Chen Zhang, et al. 2025. Debate on Graph: A Flexible and Reliable Reasoning Framework for Large Language Models. In *AAAI*.
- [27] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, et al. 2025. Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval Augmented Generation. In *ICLR*.
- [28] Costas Mavromatis et al. 2025. GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs. In *Findings of ACL*.
- [29] Mengting Pan, Fan Li, Xiaoyang Wang, Wenjie Zhang, and Xuemin Lin. 2025. HiTeC: Hierarchical Contrastive Learning on Text-Attributed Hypergraph with Semantic-Aware Augmentation. *arXiv preprint arXiv:2508.03104* (2025).
- [30] Boci Peng, Yun Zhu, Yongchao Liu, et al. 2026. Graph Retrieval-Augmented Generation: A Survey. *ACM transactions on information systems* 44 (2026), 1–52.
- [31] Paolo Rosso, Dingqi Yang, and Philippe Cudré-Mauroux. 2020. Beyond Triplets: Hyper-Relational Knowledge Graph Embedding for Link Prediction. In *WWW*.
- [32] Apoorv Saxena, Aditay Tripathi, and Partha P. Talukdar. 2020. Improving Multi-hop Question Answering over Knowledge Graphs using Knowledge Base Embeddings. In *ACL*.
- [33] Qing Sima, Jianke Yu, Xiaoyang Wang, Wenjie Zhang, Ying Zhang, and Xuemin Lin. 2025. Deep Overlapping Community Search via Subspace Embedding. *Proc. ACM Manag. Data* 3, 1 (2025), 28:1–28:26.
- [34] Jiashuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, et al. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *ICLR*.
- [35] Xingyu Tan, Jingya Qian, Chen Chen, Sima Qing, Yanping Wu, Xiaoyang Wang, and Wenjie Zhang. 2023. Higher-Order Peak Decomposition. In *CIKM*.
- [36] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, et al. 2026. PrivGemo: Privacy-Preserving Dual-Tower Graph Retrieval for Empowering LLM Reasoning with Memory Augmentation. *arXiv preprint arXiv:2601.08739* (2026).
- [37] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, and Wenjie Zhang. 2025. Paths-over-Graph: Knowledge Graph Empowered Large Language Model Reasoning. In *WWW*. ACM, 3505–3522.
- [38] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, et al. 2025. MemoTime: Memory-Augmented Temporal Knowledge Graph Enhanced Large Language Model Reasoning. *arXiv preprint arXiv:2510.13614* (2025).
- [39] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2025. HydraRAG: Structured Cross-Source Enhanced Large Language Model Reasoning. In *EMNLP*. 14442–14470.
- [40] Xushuo Tang, Liuyi Chen, et al. 2025. Tabular-textual question answering: From parallel program generation to large language models. *World Wide Web* (2025).
- [41] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.
- [42] Changjian Wang, Weihong Deng, Wei Guan, Quan Lu, and Ning Jiang. 2025. Cross-Granularity Hypergraph Retrieval-Augmented Generation for Multi-hop Question Answering. *arXiv preprint arXiv:2508.11247* (2025).
- [43] Jianwei Wang et al. 2025. LLM-based HSE Compliance Assessment: Benchmark, Performance, and Advancements. *arXiv preprint arXiv:2505.22959* (2025).
- [44] Jianmin Wang, Kai Wang, Ying Zhang, Wenjie Zhang, Xiwei Xu, and Xuemin Lin. 2025. On LLM-Enhanced Mixed-Type Data Imputation with High-Order Message Passing. *Proc. VLDB Endow.* (2025).
- [45] Yikun Wang, Dian Ouyang, Zhuoran Wang, Dong Wen, and Xuemin Lin. 2025. Efficient Route and Area Matching Query in Dynamic Road Networks. In *ICDE*.
- [46] Yiqi Wang, Long Yuan, et al. 2023. Towards Efficient Shortest Path Counting on Billion-Scale Graphs. In *ICDE*.
- [47] Yiqi Wang, Long Yuan, et al. 2024. Simpler is More: Efficient Top-K Nearest Neighbors Search on Large Road Networks. *Proc. VLDB Endow.* (2024).
- [48] Jianfeng Wen, Jianxin Li, Yongyi Mao, et al. 2016. On the Representation and Embedding of Knowledge Bases beyond Binary Relations. In *IJCAI*.
- [49] Junde Wu, Jiayuan Zhu, et al. 2025. Medical Graph RAG: Evidence-based Medical Large Language Model via Graph Retrieval-Augmented Generation. In *ACL*.
- [50] Yanping Wu, Renjie Sun, Xiaoyang Wang, Dong Wen, Ying Zhang, Lu Qin, and Xuemin Lin. 2024. Efficient Maximal Frequent Group Enumeration in Temporal Bipartite Graphs. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3243–3255.
- [51] Yanping Wu, Renjie Sun, Xiaoyang Wang, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2024. Efficient Maximal Temporal Plex Enumeration. In *ICDE*.
- [52] Yanping Wu, Renjie Sun, Xiaoyang Wang, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2025. Efficient k-Plex Mining in Temporal Graphs. *IEEE Trans. Knowl. Data Eng.* 37, 12 (2025), 7105–7119.
- [53] Yuhuan Wu, Yuanyuan Xu, et al. 2023. A Holistic Approach for Answering Logical Queries on Knowledge Graphs. In *ICDE*.
- [54] Yuhuan Wu, Yuanyuan Xu, et al. 2024. Query2GMM: Learning Representation with Gaussian Mixture Model for Reasoning over Knowledge Graphs. In *WWW*.
- [55] Peiting Xie, Xiangjun Zai, Yanping Wu, Xiaoyang Wang, et al. 2025. HL-index: Fast Reachability Query in Hypergraphs. *arXiv preprint arXiv:2512.23345* (2025).
- [56] Derong Xu, Xinhang Li, Ziheng Zhang, Zhenxi Lin, Zhihong Zhu, et al. 2025. Harnessing Large Language Models for Knowledge Graph Question Answering via Adaptive Multi-Aspect Retrieval-Augmentation. In *AAAI*.
- [57] Zian Zhai, Fan Li, Xingyu Tan, Xiaoyang Wang, and Wenjie Zhang. 2025. Graph is a Natural Regularization: Revisiting Vector Quantization for Graph Representation Learning. *arXiv preprint arXiv:2508.06588* (2025).
- [58] Zian Zhai, Qing Sima, Xiaoyang Wang, and Wenjie Zhang. 2025. SGPT: Few-Shot Prompt Tuning for Signed Graphs. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 4045–4055.
- [59] Wenqian Zhang, Zhengyi Yang, Dong Wen, et al. 2025. Accelerating Core Decomposition in Billion-Scale Hypergraphs. *Proc. ACM Manag. Data* (2025).
- [60] Qi Zhao, Hongyu Yang, Qi Song, Xinwei Yao, and Xiangyang Li. 2025. Know-Path: Knowledge-enhanced Reasoning via LLM-generated Inference Paths over Knowledge Graphs. *arXiv preprint arXiv:2502.12029* (2025).
- [61] Ruilin Zhao, Feng Zhao, Long Wang, Xianzhi Wang, and Guandong Xu. 2024. KG-CoT: Chain-of-Thought Prompting of Large Language Models over Knowledge Graphs for Knowledge-Aware Question Answering. In *IJCAI*.
- [62] Xiangrong Zhu, Yuexiang Xie, Yi Liu, Yaliang Li, and Wei Hu. 2025. Knowledge Graph-Guided Retrieval Augmented Generation. In *NAACL*.
- [63] Yifan Zhu, Hanchen Wang, et al. 2025. UniCom: Towards a Unified and Cohesiveness-aware Framework for Community Search and Detection. *arXiv preprint arXiv:2512.02460* (2025).

A Algorithm

A.1 Planning

We summarize the comprehensive algorithmic procedure of planning as shown in Algorithm 1.

Algorithm 1: PlanInit

Input : Question Subgraph $\mathcal{H}_q = (\mathcal{V}_q, \mathcal{E}_q)$, Question q , Topic Entities $\mathcal{T}$, Target Hyperedges $\mathcal{R}$, Plan Depth d_p , #Initial Plans n_0

Output : Set of initial DAGs $\mathcal{D}_0$

```

1  $\mathcal{E}_p \leftarrow \{e \mid v \in e \wedge v \in \mathcal{T}\} \cup \mathcal{R}$ ;
2  $\mathcal{V}_p \leftarrow \{v \mid v \in e \wedge e \in \mathcal{E}_p\}$ ;
3  $\mathcal{F}_e \leftarrow \{e \mid v \in e \wedge v \in \mathcal{T}\} \cup \mathcal{R}$ ;
4 for  $d \leftarrow 1$  to  $d_p$  do
5    $\mathcal{F}'_e \leftarrow \emptyset$ ;
6   for each  $e \in \mathcal{F}_e$  do
7      $F_{\text{cand}} \leftarrow \emptyset$ ;
8     for each  $e' \in \text{Nbr}(e)$  do
9        $S_{\text{max}} \leftarrow \max\{\text{SE}(v \mid q) \mid v \in V(e) \cap V(e')\}$ ;
10       $S_{\text{avg}} \leftarrow \text{mean}\{\text{SE}(v \mid q) \mid v \in V(e) \cap V(e')\}$ ;
11       $\text{SH}(e' \mid q, e) \leftarrow \text{Combine}(S_{\text{max}}, S_{\text{avg}})$ ;
12       $F_{\text{cand}} \leftarrow F_{\text{cand}} \cup \{(e', \text{SH}(e' \mid q, e))\}$ ;
13    $F_{\text{sel}} \leftarrow \text{RankSelectDirections}(F_{\text{cand}})$ ;
14    $\mathcal{F}'_e \leftarrow \mathcal{F}'_e \cup F_{\text{sel}}$ ;
15    $\mathcal{E}_p \leftarrow \mathcal{E}_p \cup F_{\text{sel}}$ ;
16    $\mathcal{V}_p \leftarrow \mathcal{V}_p \cup \{v \mid v \in e \wedge e \in F_{\text{sel}}\}$ ;
17    $\mathcal{F}_e \leftarrow \mathcal{F}'_e$ ;
18  $c_p \leftarrow \text{FormPlanContext}(\mathcal{H}_p)$ ;
19  $\mathcal{D}_0 \leftarrow \emptyset$ ;
20 for  $i \leftarrow 1$  to  $n_0$  do
21    $Q, \mathcal{L} \leftarrow \text{LLMPlan}(q, \mathcal{T}, c_p)$ ;
22    $D \leftarrow \text{HasseTopSort}(Q, \mathcal{L})$ ;
23    $\mathcal{D}_0 \leftarrow \mathcal{D}_0 \cup \{D\}$ ;
24 return  $\mathcal{D}_0$ ;
```

A.2 Reasoning

We summarize the comprehensive algorithmic procedure of reasoning as shown in Algorithm 2.

Algorithm 2: Reasoning

Input : Initial DAGs $\mathcal{D}_0$, Question Subgraph $\mathcal{H}_q$, KH Exploration Depth Limit d_{max} , Beam Width b , Max #Solutions K

Output : Completed DAGs $\mathcal{D}_{\text{comp}}$

```

1  $\mathcal{D}_{\text{comp}} \leftarrow \emptyset$ ;
2  $\mathcal{F} \leftarrow \emptyset$ ;
3 for each  $D \in \mathcal{D}_0$  do
4    $\mathcal{F} \leftarrow \mathcal{F} \cup \{(D, 0)\}$ ;
5 while  $\mathcal{F} \neq \emptyset$  and  $|\mathcal{D}_{\text{comp}}| < K$  do
6    $(D, i) \leftarrow \mathcal{F}.\text{pop}()$ ;
7   if  $i \geq |D.\text{levels}|$  then
8      $\mathcal{D}_{\text{comp}} \leftarrow \mathcal{D}_{\text{comp}} \cup \{D\}$ ;
9     continue;
10   $Q_i \leftarrow$  subquestions at level  $i$  of  $D$ ;
11   $\mathcal{AP}_i \leftarrow \emptyset$ ;
12  for each  $q_j \in Q_i$  do
13     $AP_j \leftarrow \text{RetrieveAnswersWithPaths}(\mathcal{H}_q, q_j, d_{\text{max}}, b)$ ;
14     $\mathcal{AP}_i[j] \leftarrow AP_j$ ;
15  for each combination of answers  $\mathbb{AP}$  in  $\mathcal{AP}_i$  do
16     $D_{\text{new}} \leftarrow \text{LLMGenerateNewDAG}(D, \mathbb{AP})$ ;
17     $\mathcal{F}.\text{push}((D_{\text{new}}, i + 1))$ ;
18 return  $\mathcal{D}_{\text{comp}}$ ;
```

A.3 Answer and Path Retrieval

We summarize the comprehensive algorithmic procedure of answer and path retrieval as shown in Algorithm 3.

Algorithm 3: RetrieveAnswersWithPaths

Input : Question Subgraph $\mathcal{H}_q = (\mathcal{V}_q, \mathcal{E}_q)$, Subquestion q_j , KH
 Exploration Depth Limit $d_{\max}$, Beam Width b

Output : Set of Step Answer - Reasoning Path Pair AP_j

// Graph re-anchoring

- 1 $\mathcal{T}_j \leftarrow \text{TopicEntityInit}(q_j, \mathcal{H}_q)$;
- 2 $\mathcal{R}_j \leftarrow \text{TargetHyperedgeMatch}(q_j, \mathcal{H}_q)$;
- 3 // Initialize frontier
- 4 $\mathcal{F}_e \leftarrow \emptyset$;
- 5 $\mathcal{P} \leftarrow \emptyset$;
- 6 **for each** $e \in \{e \mid v \in e \wedge v \in \mathcal{T}_j\} \cup \mathcal{R}_j$ **do**
- 7 $\mathcal{F}_e \leftarrow \mathcal{F}_e \cup \{(e, [e])\}$;
- 8 $\mathcal{P} \leftarrow \mathcal{P} \cup \{[e]\}$;
- 9 **for** $d \leftarrow 1$ **to** $d_{\max}$ **do**
- 10 // Beam search from current frontier
- 11 $F_{\text{cand}} \leftarrow \emptyset$;
- 12 **for each** $(e, p_e) \in \mathcal{F}_e$ **do**
- 13 **for each** $e' \in \text{Nbr}(e)$ **do**
- 14 $S_{\max} \leftarrow \max\{\text{EW}(v \mid q_j) \mid v \in V(e) \cap V(e')\}$;
- 15 $S_{\text{avg}} \leftarrow \text{mean}\{\text{EW}(v \mid q_j) \mid v \in V(e) \cap V(e')\}$;
- 16 $\text{EWO}(e' \mid q_j, e) \leftarrow \text{Combine}(S_{\max}, S_{\text{avg}})$;
- 17 $p_{e'} \leftarrow p_e \oplus [e']$;
- 18 $F_{\text{cand}} \leftarrow F_{\text{cand}} \cup \{(p_{e'}, \text{EWO}(e' \mid q_j, e))\}$;
- 19 $F_{\text{cand}} \leftarrow \text{RankSelectDirections}(F_{\text{cand}})$;
- 20 $F_{\text{sel}} \leftarrow \text{LLMSelectDirections}(F_{\text{cand}}, q_j, b)$;
- 21 update $\mathcal{F}_e$ and $\mathcal{P}$ with F_{sel} ;
- 22 // Form and select reasoning paths
- 23 $P_{\text{cand}} \leftarrow \text{FormPaths}(\mathcal{P})$;
- 24 $P_{\text{cand}} \leftarrow \text{RankSelectPaths}(P_{\text{cand}})$;
- 25 $P_{\text{sel}} \leftarrow \text{LLMSelectPaths}(P_{\text{cand}}, q_j)$;
- 26 **if** $P_{\text{sel}} \neq \emptyset$ **then**
- 27 // Attempt subquestion
- 28 $AP_j \leftarrow \emptyset$;
- 29 **for each** $p_j \in P_{\text{sel}}$ **do**
- 30 $c_j \leftarrow \text{KnowledgeFusion}(p_j)$;
- 31 $a_j \leftarrow \text{LLMAnswerStep}(c_j, q_j)$;
- 32 $AP_j \leftarrow AP_j \cup \{(a_j, p_j)\}$;
- 33 **return** AP_j ;
- 34 **return** $\emptyset$;

Table 6: Ablation study results on long-range multi-hop QA.

Method	Agriculture		Legal	
	F1	G-E	F1	G-E
PROH	81.01	84.18	71.93	77.68
w/o Planning	78.28	82.36	68.49	75.36
w/o Reasoning	78.12	82.39	71.13	76.48
w/o Path retrieval	69.55	75.95	65.06	71.74
w/o Retrieval	30.61	49.12	33.24	50.77

Table 7: Performance vs. state search strategy.

# Init n_0	# Soln K	BFS			DFS		
		F1	$\hat{w}_T$	V_T	F1	$\hat{w}_T$	V_T
1	1	57.86	28.00	8.31	55.52	11.83	3.29
1	2	56.93	41.37	9.70	53.22	12.88	4.50
1	3	57.70	18.58	9.24	57.16	12.65	5.67
2	2	54.80	44.62	14.02	54.98	10.21	4.77
2	3	58.58	39.13	15.15	57.15	10.04	6.24
3	3	61.94	126.85	20.56	58.48	12.25	6.58

B Additional Experiment

B.1 Ablation Study

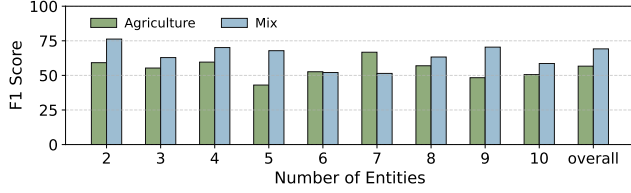
(RQ7) How do individual PROH modules affect long-range multi-hop QA performance? We conduct an ablation study on additional long-range questions from the Agriculture and Legal domains to quantify the contribution of individual modules. As shown in Table 6, removing the planning module leads to an F1 drop of up to 3.44%, indicating that context-aware structured question decomposition is necessary to divide and conquer long-range questions. Excluding the reasoning module results in an F1 decrease of up to 2.89%, suggesting that iterative refinement over reasoning plans helps correct early-stage errors. Disabling path retrieval and only retrieving anchored graph elements (shallow retrieval) causes a noticeable performance drop of up to 11.46% in F1, highlighting the importance of deep exploration over KHS to capture multi-hop relational chains for long-range QA. When the retrieval module is removed entirely, F1 drops substantially by up to 50.4%, indicating a performance collapse. Overall, these results demonstrate that each module contributes effectively to PROH’s multi-hop QA capacity.

(RQ8) How does the state search strategy affect the performance of PROH? For this experiment, we randomly sample 200 questions from the Medicine domain, and compare breadth-first (BFS) and depth-first (DFS) state search strategies under different settings of the number of initial plans n_0 and the maximum number of solutions K . As reported in Table 7, the BFS strategy consistently achieves higher F1 than the DFS strategy, however, this performance advantage comes with a significant extra computational cost. When $n_0=3$ and $K=3$, BFS exhibits explosive growth in search width $\hat{w}_T$, it also visits 20.56 states on average, which is more than 3x of DFS. DFS, though, as expected, has a much more stable width. When we fix one of n_0 or K , increasing the other will always improve the F1 score for both strategies. Overall, DFS offers a better performance-to-cost ratio and shows a more stable scaling behavior.

(RQ9) How does the plan depth affect the state search tree? As shown in Table 8, increasing the plan depth consistently improves both F1 and G-E scores. When d_p increases from 1 to 3, F1 and G-E rise from 55.65% to 59.47% and from 67.71% to 70.45%, respectively. This indicates that deeper plan depth provides more comprehensive context for planning. The average peak search tree depth $\hat{d}_T$ increases when d_p increases from 1 to 2 and then decreases to 1.36 when $d_p=3$, suggesting that, with a richer planning context, a more efficient plan can be generated. Overall, deeper plan depth d_p enhances performance without introducing excessive reasoning complexity through question decomposition.

Table 8: Performance vs. plan depth.

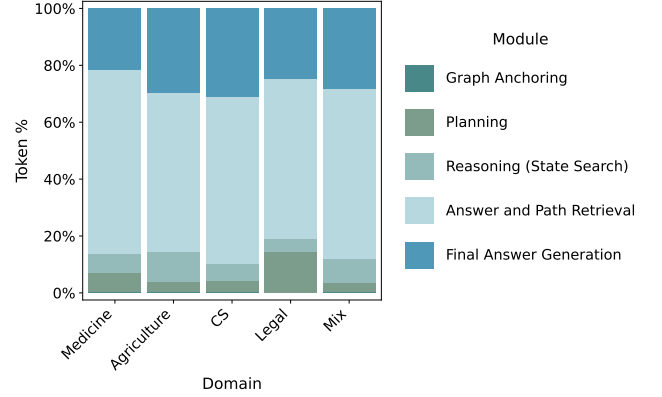
Plan depth d_p	F1	G-E	$\hat{d}_T$
1	55.65	67.71	1.38
2	57.13	68.57	1.42
3	59.47	70.45	1.36

**Figure 5: F1 score vs. #entities in ground-truth context.**

B.2 Effectiveness and Efficiency Evaluation

(RQ10) How does PROH perform with multiple entities in the ground-truth context? To evaluate the robustness PROH to question complexity with respect to entity cardinality, we categorize questions from the Agriculture and Mix domains by the number of entities appearing in the ground-truth context and report the F1 score across varying entity cardinality. Figure 5 reports the average F1 scores of PROH across different levels of relational complexity in the questions. The complexity is measured by the number of entities participating in the ground-truth context for the question. Overall, PROH exhibits stable performance as the number of entities increases, indicating robustness to growing relational complexity.

(RQ11) How does token usage distribute across modules? Figure 6 summarizes token consumption across five domains, revealing a highly skewed distribution toward the Answer and Path Retrieval module. This is largely attributable to the high token overhead of LLM-based entity scoring and path selection. Final Answer Generation remains secondary across domains, where retrieved knowledge from multiple completed reasoning DAGs is leveraged for answer generation and selection. Planning consistently ranks third, reflecting the moderate overhead of conditioning on planning context. Reasoning (i.e., research DAG refinement) consumes relatively few tokens, accounting for only a small fraction of overall token usage.

**Figure 6: Token usage among modules.**

C Experiment Details

Baselines. We compare PROH against baselines in four categories. *i) LLM-only*: Directly generates answers using the intrinsic knowledge of the LLM; *ii) StandardRAG*: A conventional chunk-based RAG approach; *iii) Graph-based RAG*: Graphs are a natural representation for modeling relational structure among entities [14, 57, 58, 63], we include two graph-based RAG methods as baselines, namely PathRAG [2] and HippoRAG2 [13]; *iv) Knowledge Hypergraph-based RAG*: Hypergraphs are widely adopted to model higher-order relationships [19, 20, 29, 55], we include the state-of-the-art KH-based RAG method HyperGraphRAG [23] as baseline. As HyperGraphRAG is the current SOTA, we directly refer to the results reported in their paper for HyperGraphRAG and other baselines in our comparisons.

Experimental Settings. Experiments are conducted using GPT-4o-mini as the primary LLM backbone, and text-embedding-3-small for vector embedding. For PROH, we set plan depth $d_p = 3$, KH exploration depth limit $d_{max} = 3$, number of initial plans $n_0 = 2$, max number of solutions $K = 2$. For PROH-L, we set plan depth $d_p = 2$, KH exploration depth limit $d_{max} = 3$, number of initial plans $n_0 = 1$, max number of solutions $K = 1$.

The complete implementation of PROH, including prompt configurations, is publicly available.¹

¹<https://github.com/zaixjun/PROH>.